

A Study on Controlling Dexterous Hands

Weikun Peng

willipwk.github.io/dex-manipulation

Abstract—Controlling a dexterous robotic hand to reproduce a human grasping demonstration is a challenging contact-rich control problem. While human videos provide a convenient source of demonstrations, retargeting only yields kinematic reference poses and does not specify the forces or torques required for stable interaction with the object. In this work, we study three representative methods for recovering such control signals from a reference hand-object interaction trajectory: differentiable trajectory optimization through a differentiable physics simulator, sampling-based Model Predictive Path Integral (MPPI) control, and reinforcement learning with a policy-gradient algorithm. We implement the three methods on the same retargeted hand-object sequence using the LEAP hand and compare them quantitatively on joint tracking, object tracking, and control effort. Our results show that differentiable simulation is sample-efficient but converges to sub-optimal controls once the task becomes contact-rich, as the gradients through contact are unreliable. MPPI and reinforcement learning produce better tracking behavior but fail to establish a stable grasp, and both require substantially more samples. We discuss the trade-offs between these methods and their suitability for offline trajectory generation versus online closed-loop control.

Index Terms—Dexterous Manipulation, Differentiable Simulation, MPPI, Reinforcement Learning

I. INTRODUCTION

Dexterous manipulation is a long-standing goal in robotics. A multi-fingered hand that can grasp, reorient, and manipulate arbitrary objects would enable robots to operate in human environments, perform fine assembly tasks, and use tools designed for human hands. However, controlling such high-degree-of-freedom systems is notoriously difficult: the dynamics involve rich, intermittent contact between many fingers and the object, the state and action spaces are high-dimensional, and small errors in finger placement or applied force can cause a grasp to slip or fail entirely.

Learning from human demonstration is an attractive path toward dexterous skills because humans routinely perform the tasks we wish to transfer to robots. A common pipeline first detects and tracks human hands in video, and then retargets the observed human hand poses onto a robotic hand. While this pipeline produces kinematically plausible reference trajectories, the retargeted poses are purely geometric: they describe where each finger should be but say nothing about the forces and torques required to actually hold the object during contact. As a result, simply tracking the reference poses on a physics-based robot typically fails to reproduce the demonstrated interaction, because the fingers pass through or bounce off the object instead of forming a stable grasp.

This project studies how to bridge this gap by computing a physical control policy for a dexterous hand given a sequence

of reference hand poses and the demonstrated object trajectory. We compare three methods that span the main families of approaches used in robotic manipulation: (i) differentiable trajectory optimization, which exploits analytical gradients through a differentiable physics simulator; (ii) Model Predictive Path Integral (MPPI) control, a sampling-based receding-horizon method that does not rely on gradients through contact; and (iii) reinforcement learning, which trains a feedback policy from interaction with a non-differentiable simulator. Each method is applied to the same retargeted hand-object sequence with the LEAP hand and evaluated on joint tracking, object tracking, and control effort.

In this project, We provide a quantitative comparison of these methods on a hand-object interaction sequence in DexYCB dataset [1] and characterize their failure modes. We analyze the trade-offs between the three approaches and discuss when each is most appropriate, with differentiable simulation best suited to offline trajectory generation and MPPI and reinforcement learning better aligned with online closed-loop control.

II. RELATED WORKS

a) Dexterous manipulation: Dexterous manipulation is a very challenging problem in robotic manipulation due to its complex dynamics and high degree of freedom action space. A majority of work studies grasp pose synthesis on the robot side, such as D(R, O) grasp [2], T(R, O) grasp [3], D-REX [4], SimToolReal [5], etc.

b) Differentiable simulation for manipulation.: Differentiable physics simulators such as Nimble Physics [6], Brax [7], expose analytic gradients of simulation outputs with respect to controls, enabling gradient-based trajectory optimization. These methods are sample-efficient when the gradients are well-behaved, but prior work has observed that gradients through contact can be noisy or biased, which limits their effectiveness in contact-rich settings [8]. Besides, previous studies also indicate that differentiable simulation may converge to sub-optimal solutions [9].

c) Sampling-based and learning-based control.: MPPI is a sampling-based model predictive control method that avoids differentiating through dynamics by scoring randomly perturbed control sequences and updating the nominal control with a softmax-weighted average. Gradient-free optimization methods are not affected by incorrect gradients but require many parallel rollouts per step. Reinforcement learning, and in particular on-policy policy-gradient algorithms such as PPO [10], offers an alternative that produces a closed-loop

feedback policy but typically requires millions of environment steps to train.

III. METHOD

We study the problem of controlling a dexterous robotic hand to reproduce a human-object grasping demonstration. The input to the system is a sequence of hand poses obtained by retargeting human hand motion to the robot hand, together with the observed object trajectory. The goal is to find physically valid finger controls that cause the simulated robot hand to maintain contact with the object and induce object motion consistent with the demonstration.

Let q_t^{ref} denote the retargeted robot hand pose at time t , and let $x_t^{\text{obj,ref}}$ denote the corresponding object pose. The robot hand state is decomposed into a global hand pose and actuated finger joints. In the methods considered here, the global hand pose is prescribed by the retargeted trajectory, while the finger joints are controlled through either position targets or torques. This isolates the dexterous grasping problem: given the demonstrated hand motion, the controller must discover finger actions that generate appropriate contact forces on the object.

A. Differentiable Trajectory Optimization

The first approach formulates grasp control as direct trajectory optimization through a differentiable physics simulator. The decision variables are the finger controls over a finite horizon,

$$U = \{u_0, u_1, \dots, u_{T-1}\}. \quad (1)$$

Starting from the initial demonstrated hand-object state, the simulator rolls the system forward under the candidate controls. Because the simulator exposes gradients of the final loss with respect to the controls, the control sequence can be optimized by gradient descent.

For a short-horizon grasp-stability setting, the loss penalizes unintended object displacement and object rotation over the rollout, together with control magnitude. This tests whether a local finger-control sequence can maintain a stable grasp under contact dynamics. For the full hand-object sequence, the loss is accumulated across the trajectory:

$$J(U) = \sum_{t=1}^T \left(w_x \|p_t^{\text{obj}} - p_t^{\text{obj,ref}}\|^2 + w_r \|r_t^{\text{obj}} - r_t^{\text{obj,ref}}\|^2 \right) + w_u \sum_{t=0}^{T-1} \|u_t\|^2. \quad (2)$$

Here p_t^{obj} and r_t^{obj} are the simulated object position and orientation. The hand base follows the retargeted trajectory kinematically, while the fingers and object evolve through physics. The optimizer updates the finger control sequence to minimize object tracking error under contact.

In practice, because gradient-based optimization is sensitive to initialization, we divide the procedure into two stages. In the first stage, the optimizer considers only hand-motion tracking; the resulting control sequence is then used as the initialization

for the second stage, which jointly optimizes hand tracking and object grasping.

B. Model Predictive Path Integral Control

The second approach uses Model Predictive Path Integral (MPPI) control. MPPI is a sampling-based trajectory optimization method that does not require differentiating through contact. Instead, it repeatedly samples perturbations around a nominal control sequence, rolls out these candidates in simulation, evaluates their costs, and updates the nominal sequence using cost-weighted averaging.

At each time step, MPPI maintains a horizon-length control sequence

$$U = \{u_t, u_{t+1}, \dots, u_{t+H-1}\}. \quad (3)$$

It samples N perturbed sequences,

$$U^{(i)} = U + \epsilon^{(i)}, \quad \epsilon^{(i)} \sim \mathcal{N}(0, \Sigma), \quad (4)$$

and evaluates each sequence by rolling it out in parallel simulation. Each rollout is scored using a cost containing joint-tracking, object-tracking, and contact-proximity terms:

$$S^{(i)} = \sum_{\tau=t}^{t+H-1} \left(w_q \|q_\tau^{(i)} - q_\tau^{\text{ref}}\|^2 + w_x \|p_\tau^{\text{obj},(i)} - p_\tau^{\text{obj,ref}}\|^2 + w_c d_{\text{contact}}^{(i)} + w_u \|u_\tau\|^2 \right). \quad (5)$$

Besides the loss on reference hand poses and object trajectory, we add a contact loss to encourage the finger tips contacting the object, as shown below.

$$d_{\text{contact}} = \frac{1}{N_{\text{finger}}} \sum_{i=1} N_{\text{finger}} \|p^{\text{tips}} - p^{\text{obj}}\|_2 \quad (6)$$

The nominal sequence is updated by a softmax-weighted average of the sampled perturbations:

$$U \leftarrow U + \sum_i \frac{\exp(-S^{(i)}/\lambda)}{\sum_j \exp(-S^{(j)}/\lambda)} \epsilon^{(i)}. \quad (7)$$

Only the first action is executed, after which the horizon is shifted forward, and the procedure repeats. This receding-horizon structure allows MPPI to adapt the planned control sequence to the current simulated hand-object state.

C. Reinforcement Learning

The third approach learns a feedback policy using reinforcement learning. Instead of optimizing a separate action sequence for each demonstration rollout, the method trains a policy

$$\pi_\theta(u_t | o_t) \quad (8)$$

that maps the current observation o_t to a finger torque command u_t . The observation includes the current finger joint state, the reference finger pose, relative fingertip-object geometry, object tracking error, and phase within the demonstration. This gives the policy information about both the local contact state and progress through the reference trajectory.

The policy is trained with a reward shaped to match the same control objective used by the optimization methods. A typical reward has the form

$$r_t = \exp(-w_q \|q_t - q_t^{\text{ref}}\|^2) + \exp(-w_x \|p_t^{\text{obj}} - p_t^{\text{obj,ref}}\|^2) + \exp(-w_c d_{\text{contact}}) - w_u \|u_t\|^2. \quad (9)$$

The exponential terms provide bounded positive rewards for matching the reference hand pose, matching the object trajectory, and maintaining fingertip proximity. The effort penalty discourages excessive torque. During training, many simulation environments are rolled out in parallel, and policy parameters are updated with an on-policy policy-gradient algorithm.

IV. EXPERIMENT

A. Implementation Details

For simplicity, we directly use the ground-truth human hand poses for retargeting. We retarget a hand-object interaction sequence from the DexYCB dataset [1] using dex-retargeting [11], and use the LEAP hand [12] as the target dexterous hand. Because many mainstream physics simulators are not differentiable, we employ different simulators depending on the method: Nimble Physics [6] for differentiable trajectory optimization, and Genesis [13] for MPPI and reinforcement learning to enable fast parallel sample collection. The reinforcement learning policy is trained with PPO [10]. For differentiable simulation, we run three independent optimization trials and keep the best; each trial performs 500 iterations. For MPPI, each planning step samples 2048 trajectories with a horizon of 30 steps. For PPO, we train for 1000 episodes.

B. Experiment Results

We compare the three methods on the task of controlling the LEAP hand to grasp an object given the retargeted reference hand poses and object trajectory. Qualitative retargeting results are shown in the first and second rows of fig. 1, and a quantitative comparison is presented in fig. 2.

Among the three methods, MPPI achieves the lowest tracking error, while differentiable simulation performs the worst. We attribute this to the contact-rich nature of the task: the gradients computed by the differentiable simulator through intermittent contact can be inaccurate or poorly conditioned, making them unreliable as descent directions. As shown in fig. 3, the optimization loss does decrease, yet the resulting control sequence is sub-optimal. Consistently, fig. 2 shows that when the optimizer is asked only to track the reference motion (without contact with the object), the policy follows the reference well; once the object is introduced, the tracking error grows sharply.

The reinforcement learning policy performs slightly worse than MPPI. We hypothesize that this is largely due to under-training: PPO typically requires on the order of one million environment steps to converge, whereas our training budget is substantially smaller. As shown in fig. 4, the training reward has not yet plateaued at the end of our training run.

V. CONCLUSION

This project compares three representative methods for a contact-rich dexterous control problem: differentiable trajectory optimization, MPPI, and reinforcement learning. Each method exhibits distinct trade-offs. Differentiable simulation is highly sample-efficient, but struggles with contact-rich dynamics because the gradients through contact are unreliable and often lead the optimizer to sub-optimal solutions. MPPI and reinforcement learning require substantially more samples, particularly PPO, but recover better control policies in the presence of contact. In addition, reinforcement learning naturally produces closed-loop policies, whereas differentiable trajectory optimization yields an open-loop control sequence. These properties suggest complementary use cases: differentiable simulation is better suited to offline applications such as generating demonstration trajectories in simulation, while reinforcement learning is more appropriate for online, closed-loop control.

Human
Demonstration



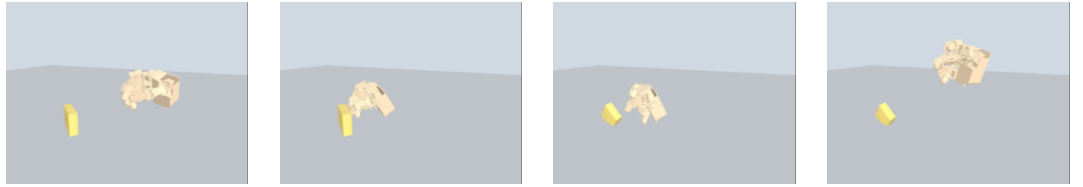
Retarget
Motion



Diffsim
w/o object



Diffsim
w/ object



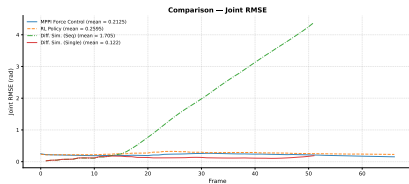
MPPI



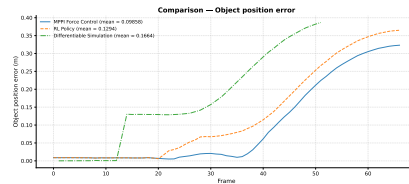
RL



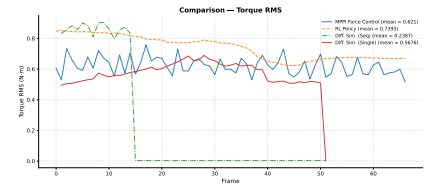
Fig. 1: Qualitative results of retargeting and the resulting control policies. None of the methods succeeds in grasping the object, even though the hand poses remain close to the reference motion.



(a) Joint error at each time step.

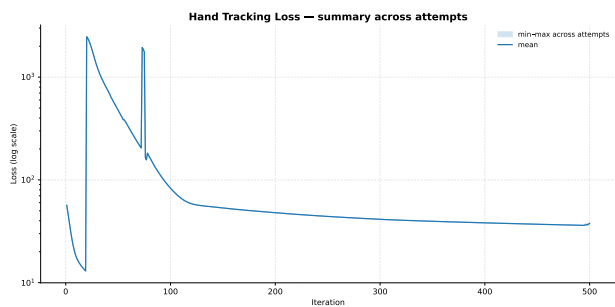


(b) Object position error at each time step.

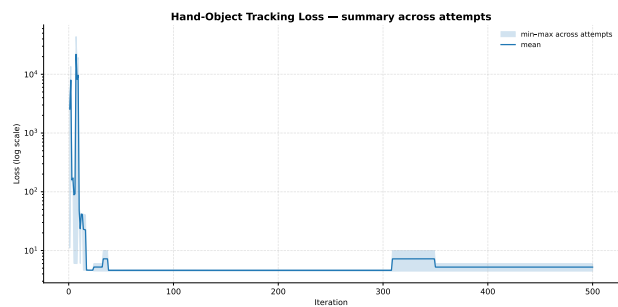


(c) Root mean square of the torque applied to the leap hand.

Fig. 2: Per-time-step quantitative results. Before contact, all methods track the reference hand motion fairly accurately. After contact, differentiable simulation fails to find a control policy that grasps the object; MPPI and RL track the reference more closely but also fail to establish a stable grasp.



(a) Loss curve for the hand-tracking stage.



(b) Loss curve for the joint hand-tracking and object-grasping stage.

Fig. 3: Loss curves for the two-stage optimization with differentiable simulation. Although the loss decreases in both stages, the previous quantitative results show that the optimizer converges to a sub-optimal solution for this contact-rich control problem.

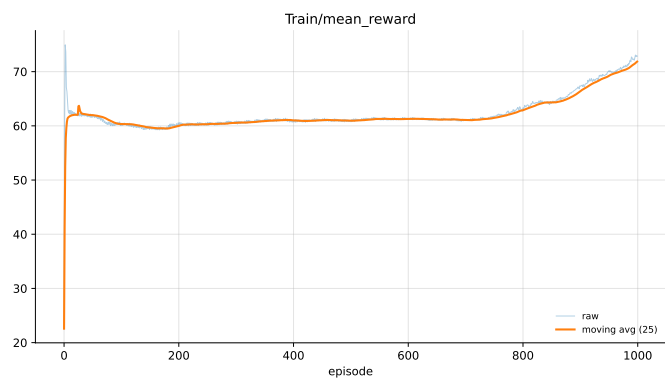


Fig. 4: Training reward curve for the reinforcement learning policy.

REFERENCES

- [1] Y.-W. Chao, W. Yang, Y. Xiang, P. Molchanov, A. Handa, J. Tremblay, Y. S. Narang, K. Van Wyk, U. Iqbal, S. Birchfield, J. Kautz, and D. Fox, “DexYCB: A benchmark for capturing hand grasping of objects,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [2] Z. Wei, Z. Xu, J. Guo, Y. Hou, C. Gao, Z. Cai, J. Luo, and L. Shao, “ $\mathcal{D}(\mathcal{R}, \mathcal{O})$ grasp: A unified representation of robot and object interaction for cross-embodiment dexterous grasping,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 4982–4988.
- [3] X. Fei, Z. Xu, H. Fang, T. Zhang, and L. Shao, “T(r,o) grasp: Efficient graph diffusion of robot-object spatial transformation for cross-embodiment dexterous grasping,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.12724>
- [4] H. Lou, M. Zhang, H. Geng, H. Zhou, S. He, Z. Gao, S. Zhao, J. Mao, P. Abbeel, J. Malik *et al.*, “D-rer: Differentiable real-to-sim-to-real engine for learning dexterous grasping,” in *The Fourteenth International Conference on Learning Representations*, 2026.
- [5] K. Kedia, T. G. W. Lum, J. Bohg, and C. K. Liu, “Simtoolreal: An object-centric policy for zero-shot dexterous tool manipulation,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.16863>
- [6] K. Werling, D. Omens, J. Lee, I. Exarchos, and C. K. Liu, “Fast and feature-complete differentiable physics for articulated rigid bodies with contact,” *arXiv preprint arXiv:2103.16021*, 2021.
- [7] C. D. Freeman, E. Frey, A. Raichuk, S. Girgin, I. Mordatch, and O. Bachem, “Brax—a differentiable physics engine for large scale rigid body simulation,” *arXiv preprint arXiv:2106.13281*, 2021.
- [8] H. J. Suh, M. Simchowitz, K. Zhang, and R. Tedrake, “Do differentiable simulators give better policy gradients?” in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, and S. Sabato, Eds., vol. 162. PMLR, 17–23 Jul 2022, pp. 20 668–20 696. [Online]. Available: <https://proceedings.mlr.press/v162/suh22b.html>
- [9] R. Antonova, J. Yang, K. M. Jatavallabhula, and J. Bohg, “Rethinking optimization with differentiable simulation from a global perspective,” in *Proceedings of The 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, K. Liu, D. Kulic, and J. Ichnowski, Eds., vol. 205. PMLR, 14–18 Dec 2023, pp. 276–286. [Online]. Available: <https://proceedings.mlr.press/v205/antonova23a.html>
- [10] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [11] Y. Qin, W. Yang, B. Huang, K. Van Wyk, H. Su, X. Wang, Y.-W. Chao, and D. Fox, “Anyteleop: A general vision-based dexterous robot arm-hand teleoperation system,” in *Robotics: Science and Systems*, 2023.
- [12] K. Shaw, A. Agarwal, and D. Pathak, “Leap hand: Low-cost, efficient, and anthropomorphic hand for robot learning,” *Robotics: Science and Systems (RSS)*, 2023.
- [13] G. Authors, “Genesis: A generative and universal physics engine for robotics and beyond,” December 2024. [Online]. Available: <https://github.com/Genesis-Embodied-AI/Genesis>